

```
"""
```

بوت "شيكي" - بوت إدارة جروب تليجرام

```
-----
```

المميزات:

1. أمر "شيكي [عدد الدقائق]" ك رد على رسالة عضو -> المدة المطلوبة (المشرفين بس) (mute) يحظره
2. قفل الجروب تلقائيًا في وقت محدد وفتحه في وقت ثاني (يومي)
3. لو حد كتب سب/إهانة للبوت -> يرد برسالة هزار ويحظره ربع ساعة تلقائيًا

المكتبات المطلوبة:

```
pip install python-telegram-bot --
upgrade
```

ملحوظة مهمة: لازم البوت يكون ****أدمن**** في الجروب وعنده صلاحية

عشان يقدر يحظر/يقفل "Restrict Members"

```
"""
```

```
import json
import logging
import os
import re
from datetime import time as dtime,
timedelta

from telegram import Update,
ChatPermissions
from telegram.ext import (
```

```

        ApplicationBuilder,
        CommandHandler,
        MessageHandler,
        ContextTypes,
        filters,
    )

# ----- الإعدادات -----
---
TELEGRAM_BOT_TOKEN =
os.environ.get("TELEGRAM_BOT_TOKEN",
"ضع_التوكن_هنا")
SETTINGS_FILE = "group_settings.json"

# كلمات لو اتقالت للبوت هيتعامل معاها كإهانة (زود عليها
براحتك)
INSULT_WORDS = ["", "احمق", "حيوان", "خرا", "غبي",
""غبيه", "زبالة", "كلب", "حمار"]

# ردود عشوائية لما حد يهين البوت
INSULT_REPLIES = [
    "", "الله يسامحك، ما كان هيك العشم منك",
    "", "طيب يا سيدي، اتحطرت ربع ساعة تستريح فيها",
    "", "ماشي، هرجع أرد عليك بعد ما تهدى شوية",
    "", "خلاص متزعلش، بس هتقعده برا شوية",
]

# مدة الحظر التلقائي عند السب (بالدقائق)
AUTO_MUTE_MINUTES = 15

logging.basicConfig(level=logging.INFO)

# ----- حفظ/تحميل إعدادات كل جروب -----

```

```

-----
def load_settings() -> dict:
    if os.path.exists(SETTINGS_FILE):
        try:
            with open(SETTINGS_FILE, "r",
encoding="utf-8") as f:
                return json.load(f)
            except Exception as e:
                logging.error(f"فشل تحميل الإعدادات:
{e}")
            return {}

def save_settings(settings: dict):
    try:
        with open(SETTINGS_FILE, "w",
encoding="utf-8") as f:
            json.dump(settings, f,
ensure_ascii=False, indent=2)
        except Exception as e:
            logging.error(f"فشل حفظ الإعدادات:
{e}")

group_settings = load_settings()

# ----- أدوات مساعدة -----
-----
async def is_admin(update: Update, context:
ContextTypes.DEFAULT_TYPE, user_id: int) ->
bool:
    try:
        member = await

```

```

context.bot.get_chat_member(update.effective_
e_chat.id, user_id)
        return member.status in
("administrator", "creator")
    except Exception:
        return False

async def mute_user(context:
ContextTypes.DEFAULT_TYPE, chat_id: int,
user_id: int, minutes: int):
    until = dt.datetime.fromtimestamp(0) #
placeholder, real calc below
    from dt.datetime import datetime
    until_date = datetime.now() +
timedelta(minutes=minutes)
    await context.bot.restrict_chat_member(
        chat_id=chat_id,
        user_id=user_id,

permissions=ChatPermissions(can_send_messag
es=False),
        until_date=until_date,
    )

# ----- أمر "شيكي" (حظر مؤقت من
المشرف) -----
async def sheky_command(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    message = update.effective_message

    # لازم يكون رد على رسالة عضو معين
    if not message.reply_to_message:

```

```

        await message.reply_text("لازم تعمل رد"
على رسالة العضو اللي عايز تحظره وتكتب: شيكي (reply)
30")

        return

    # لازم يكون مشرف
    if not await is_admin(update, context,
update.effective_user.id):
        await message.reply_text("الأمر ده
😊 للمشرفين بس")
        return

    target_user =
message.reply_to_message.from_user

    # استخراج عدد الدقائق من النص (لو موجود)،
    الافتراضي 30 دقيقة
    match = re.search(r"شيكي\s+(\d+)",
message.text)
    minutes = int(match.group(1)) if match
    else 30

    try:
        await mute_user(context,
update.effective_chat.id, target_user.id,
minutes)
        await message.reply_text(
            f"تم حظر"
{target_user.first_name} لمدة {minutes} دقيقة
🚫"
        )
    except Exception as e:
        logging.error(f"فشل الحظر: {e}")
        await message.reply_text("مقدرتش")

```

Restrict أحظره، تأكد إنني أؤمن وعندي صلاحية Members.")

----- الرد التلقائي على الإهانة -----

```
async def handle_message(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    message = update.effective_message
    if not message or not message.text:
        return
```

```
    text = message.text.strip()
```

لو الرسالة فيها كلمة "شيكي" ومعناها رقم، سيبها لهاندلر الأمر الثاني

```
    if text.startswith("شيكي"):
        return
```

هل الرسالة موجهة للبوت وفيها سب؟

```
    is_reply_to_bot = (
        message.reply_to_message
        and
        message.reply_to_message.from_user
        and
        message.reply_to_message.from_user.is_bot
    )
```

```
    contains_insult = any(word in text for
word in INSULT_WORDS)
```

```
    if contains_insult and (is_reply_to_bot
or "بوت" in text or "شيكي" in text):
        import random
```

```

        reply =
random.choice(INSULT_REPLIES)
        try:
            await mute_user(
                context,
update.effective_chat.id,
update.effective_user.id, AUTO_MUTE_MINUTES
            )
            await message.reply_text(reply)
        except Exception as e:
            logging.error(f"فشل الحظر التلقائي:
{e}")
            await message.reply_text(reply
+ "\n(هنا أدمن مش أخطرلك، مش أخطرلك 😊)")

# ----- قفل/فتح الجروب التلقائي -----
-----
async def lock_group(context:
ContextTypes.DEFAULT_TYPE):
    chat_id = context.job.chat_id
    try:
        await
context.bot.set_chat_permissions(
            chat_id,
ChatPermissions(can_send_messages=False)
        )
        await
context.bot.send_message(chat_id, "🔒 الجروب
المتقفل دلوقتي، هيتفتح تاني في الميعاد المحدد
")
    except Exception as e:
        logging.error(f"فشل قفل الجروب: {e}")

```

```

async def unlock_group(context:
ContextTypes.DEFAULT_TYPE):
    chat_id = context.job.chat_id
    try:
        await
context.bot.set_chat_permissions(
    chat_id,
    ChatPermissions(can_send_messages=True)
    )
        await
context.bot.send_message(chat_id, "🔓 الجروب
اتفضلوا، افتتح، 🎉")
    except Exception as e:
        logging.error(f"فشل فتح الجروب: {e}")

async def schedule_lock(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    """
    الاستخدام: /جدولة_القفل 08:00 23:00
    يعني يقفل الجروب الساعة 11 بالليل، ويفتحة الساعة 8
    (الصبح، يوميًا)
    """
    message = update.effective_message

    if not await is_admin(update, context,
update.effective_user.id):
        await message.reply_text("الأمر ده
😏 للمشرفين بس")
        return

    args = context.args
    if len(args) != 2:

```



```

        await message.reply_text(
            "استخدم الأمر كده: /جدولة_القفل 23:00
08:00\n"
            "(وقت القفل، متبوع بوقت الفتح)"
        )
        return

    try:
        lock_h, lock_m = map(int,
args[0].split(":"))
        unlock_h, unlock_m = map(int,
args[1].split(":"))
    except ValueError:
        await message.reply_text("الصيغة غلط،
الصيغة غلط،
HH:MM 23:00 لازم تكون
")
        return

    chat_id = update.effective_chat.id

    # نسمح أي جدولة قديمة لنفس الجروب الأول
    current_jobs =
context.job_queue.get_jobs_by_name(f"lock_{
chat_id}") + \

context.job_queue.get_jobs_by_name(f"unlock
_{chat_id}")
    for job in current_jobs:
        job.schedule_removal()

    context.job_queue.run_daily(
        lock_group, time=datetime(hour=lock_h,
minute=lock_m), chat_id=chat_id,
name=f"lock_{chat_id}"
    )

```

```

context.job_queue.run_daily(
    unlock_group,
    time=dttime(hour=unlock_h, minute=unlock_m),
    chat_id=chat_id, name=f"unlock_{chat_id}"
)

group_settings[str(chat_id)] = {
    "lock_time": args[0],
    "unlock_time": args[1],
}
save_settings(group_settings)

await message.reply_text(
    f"تم ✅ الجروب هيتقفل الساعة {args[0]}
    يومياً {args[1]} وهيتفتح الساعة
    )

async def start(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "🤖 أهلاً! أنا بوت شيكي"
        "الأوامر المتاحة:\n"
        "رد على أي عضو واكتب: شيكي 30 (لحظره 30\n"
        "جدولة_القفل 08:00 23:00 (لجدولة قفل // •"
        "\nفتح الجروب يومياً)"
        "لازم تخليبي أدمن في الجروب وعندي صلاحية"
        "عشان أشتغل صح Restrict Members"
    )

# ----- إعادة تحميل الجدولة القديمة عند -----
# ----- تشغيل البوت -----

```

```

def restore_schedules(app):
    for chat_id_str, settings in
group_settings.items():
        chat_id = int(chat_id_str)
        lock_h, lock_m = map(int,
settings["lock_time"].split(":"))
        unlock_h, unlock_m = map(int,
settings["unlock_time"].split(":"))
        app.job_queue.run_daily(
            lock_group,
time=dttime(hour=lock_h, minute=lock_m),
chat_id=chat_id, name=f"lock_{chat_id}"
        )
        app.job_queue.run_daily(
            unlock_group,
time=dttime(hour=unlock_h, minute=unlock_m),
chat_id=chat_id, name=f"unlock_{chat_id}"
        )

def main():
    app =
ApplicationBuilder().token(TELEGRAM_BOT_TOK
EN).build()

    app.add_handler(CommandHandler("start",
start))

app.add_handler(CommandHandler("جدولة_القفل",
schedule_lock))

app.add_handler(MessageHandler(filters.Rege
x(r"^شيكِي(\s+\d+)?$", sheky_command))

```

```
app.add_handler(MessageHandler(filters.TEXT
& ~filters.COMMAND, handle_message))

restore_schedules(app)

print("عشان Ctrl+C بوت شيكي شغال ... اضغط")
app.run_polling()

if __name__ == "__main__":
    main()
```